

GBR Übung 04

[PDF](#)

4/4 4.1: Syscall

2/2 a)

C-Bibliothek:

```
#include <sys/time.h>

int main() {
    struct timeval tv;
    for(int i=0; i<100000000; i++) {
        gettimeofday(&tv, NULL); ✓
    }
    return 0;
}
```

Syscall:

```
#include <sys/syscall.h>
#include <unistd.h>
#include <sys/time.h>

int main() {
    struct timeval tv;
    for(int i=0; i<100000000; i++) {
        syscall(SYS_gettimeofday, &tv, NULL); ✓
    }
    return 0;
}
```

1/1 b)

```
• > time ./libc  
./libc 0,14s user 0,00s system 99% cpu 0,138 total  
• > time ./syscall  
./syscall 0,23s user 0,58s system 99% cpu 0,819 total
```

- Das lib-Programm ist deutlich schneller als die direkte syscall-Variante
- Bei lib fällt keine Systemzeit an 
- Beim direkten syscall dominiert die Systemzeit.

-> gettimeofday führt keinen System Aufruf aus

1/1 c)

lib

```
Breakpoint 1, 0x00007ffff7fc4880 in gettimeofday ()
(gdb) step
Single stepping until exit from function gettimeofday,
which has no line number information.
0x000055555555509a in main ()
(gdb) dis
disable      disassemble  disconnect   display
(gdb) dis
disable      disassemble  disconnect   display
(gdb) disassemble
Dump of assembler code for function main:
    0x0000555555555060 <+0>:      push    %rbp
    0x0000555555555061 <+1>:      mov     %rsp,%rbp
    0x0000555555555064 <+4>:      push    %r12
    0x0000555555555066 <+6>:      push    %rbx
    0x0000555555555067 <+7>:      lea     -0x30(%rbp),%r12
    0x000055555555506b <+11>:     sub     $0x20,%rsp
    0x000055555555506f <+15>:     mov     %fs:0x28,%rbx
    0x0000555555555078 <+24>:     mov     %rbx,-0x18(%rbp)
    0x000055555555507c <+28>:     mov     $0x989680,%ebx
    0x0000555555555081 <+33>:     data16 cs nopw 0x0(%rax,%rax,1)
    0x000055555555508c <+44>:     nopl    0x0(%rax)
    0x0000555555555090 <+48>:     xor     %esi,%esi
    0x0000555555555092 <+50>:     mov     %r12,%rdi
    0x0000555555555095 <+53>:     call    0x555555555040 <gettimeofday@plt>
⇒ 0x000055555555509a <+58>:     sub     $0x1,%ebx
    0x000055555555509d <+61>:     jne     0x555555555090 <main+48>
    0x000055555555509f <+63>:     mov     -0x18(%rbp),%rax
    0x00005555555550a3 <+67>:     sub     %fs:0x28,%rax
    0x00005555555550ac <+76>:     jne     0x5555555550b9 <main+89>
    0x00005555555550ae <+78>:     add     $0x20,%rsp
    0x00005555555550b2 <+82>:     xor     %eax,%eax
    0x00005555555550b4 <+84>:     pop     %rbx
    0x00005555555550b5 <+85>:     pop     %r12
    0x00005555555550b7 <+87>:     pop     %rbp
    0x00005555555550b8 <+88>:     ret
    0x00005555555550b9 <+89>:     call    0x555555555030 <__stack_chk_fail@plt>
End of assembler dump.
```



syscall

```
Breakpoint 1 at 0x1064
(gdb) run
Starting program: /home/joshua/Documents/Studium/25-26_WiSe/Betriebssysteme-und-F
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/xx7cm72qy2c0643cm1ipngd87aqwkcdp-gli

Breakpoint 1, 0x0000555555555064 in main ()
(gdb) disassemble
Dump of assembler code for function main:
   0x0000555555555060 <+0>:      push    %rbp
   0x0000555555555061 <+1>:      mov     %rsp,%rbp
⇒ 0x0000555555555064 <+4>:      push    %r12
   0x0000555555555066 <+6>:      push    %rbx
   0x0000555555555067 <+7>:      lea     -0x30(%rbp),%r12
   0x000055555555506b <+11>:     sub     $0x20,%rsp
   0x000055555555506f <+15>:     mov     %fs:0x28,%rbx
   0x0000555555555078 <+24>:     mov     %rbx,-0x18(%rbp)
   0x000055555555507c <+28>:     mov     $0x989680,%ebx
   0x0000555555555081 <+33>:     data16 cs nopw 0x0(%rax,%rax,1)
   0x000055555555508c <+44>:     nopl    0x0(%rax)
   0x0000555555555090 <+48>:     xor     %edx,%edx
   0x0000555555555092 <+50>:     xor     %eax,%eax
   0x0000555555555094 <+52>:     mov     %r12,%rsi
   0x0000555555555097 <+55>:     mov     $0x60,%edi
   0x000055555555509c <+60>:     call    0x555555555040 <syscall@plt>
   0x00005555555550a1 <+65>:     sub     $0x1,%ebx
   0x00005555555550a4 <+68>:     jne     0x555555555090 <main+48>
   0x00005555555550a6 <+70>:     mov     -0x18(%rbp),%rax
   0x00005555555550aa <+74>:     sub     %fs:0x28,%rax
   0x00005555555550b3 <+83>:     jne     0x5555555550c0 <main+96>
   0x00005555555550b5 <+85>:     add     $0x20,%rsp
   0x00005555555550b9 <+89>:     xor     %eax,%eax
   0x00005555555550bb <+91>:     pop     %rbx
   0x00005555555550bc <+92>:     pop     %r12
   0x00005555555550be <+94>:     pop     %rbp
   0x00005555555550bf <+95>:     ret
   0x00005555555550c0 <+96>:     call    0x555555555030 <__stack_chk_fail@plt>
End of assembler dump.
```



Der Unterschied wird sichtbar bei libc +53 und syscall +60.

4.2:

a)

- Ausgabe 15:
 - `t1_main` ruft `t1_f1` auf. `x=3` wird auf den Stack gelegt (Offset `-0x4(%rbp)`). `p` wird auf die Adresse von `x` gesetzt. `t1_f1` kehrt zurück.

- `t1_main` ruft `t1_f2` auf. Der Stack-Frame wird an derselben Stelle wie für `f1` erstellt. `y=5` wird an dieselbe relative Adresse gelegt (`-0x4(%rbp)`).
- Gleichzeitig läuft `t2_main . sleep(1)`, was T1 Zeit gibt, `p` zu setzen und `k` zu erhöhen (`k` wird 3).
- T2 dereferenziert `*p`. Da `p` auf die Adresse `-0x4(%rbp)` zeigt (wo früher `x` war und jetzt `y` ist), liest es den Wert 5.
- $k = k * (*p) \rightarrow 3 * 5 = 15$.
- 0, Zufallswerte oder Segfault:
 - Wenn T2 liest, nachdem `t1_f2` bereits zurückgekehrt ist, zeigt `p` in einen "ungültigen" Stackbereich unterhalb des aktuellen Stackpointers (oder in den Frame von `t1_main`). Der Wert ist dann Garbage.
 - Wenn T2 extrem schnell wäre (vor T1s `p=&x`), wäre `p` nicht initialisiert (Global `int* p` ist im BSS 0-initialisiert). Dereferenzierung von `NULL` führt zum Absturz (Segmentation Fault).

b)

Das Verhalten ist undefiniert, da ein "Dangling Pointer" entsteht, weil die globale Variable `p` mit der Adresse der lokalen Variable `x` belegt wird.

- Sobald `t1_f1` endet (return), wird der Speicherbereich von `x` auf dem Stack freigegeben.
- Der Zugriff auf diesen Speicherbereich durch einen anderen Thread (`t2_main`) ist ein klassischer "Use-after-free"-Fehler bzw. Zugriff auf verwaisten Speicher.

4.3 1,5/4

a) 1/2

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
ZS 1	R1	1. DOWN(mutex_str)	mutex_str = 0	OK
		2. DOWN(sem_r)	sem_r = 0	OK
		3. DOWN(mutex_rc)	mutex_rc = 0	OK
ZS 2**	R1	4. readcount++	readcount = 1	
		5. if (readcount == 1) -> DOWN(sem_w)	sem_w = 0	Writer blockiert
		6. UP(mutex_rc)	mutex_rc = 1	
Interrupt		ankunft R2 und W1		
ZS 3	R2	1. DOWN(mutex_str)		Blockiert
ZS 4	W1	1. DOWN(mutex_wc)	mutex_wc = 0	OK
		2. writecount++	writecount = 1	
		3. if (writecount == 1) -> DOWN(sem_r)		Blockiert

Wie sieht der weitere Ablauf aus, bis alle Prozesse fertig sind? -1

=> R1 hält mutex_str und sem_r. R2 wartet auf mutex_str. W1 wartet auf sem_r.

b) 0,5/2

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
ZS 1	W1	1. DOWN(mutex_wc)	mutex_wc = 0	OK
		2. writecount++	writecount = 1	
		3. if (writecount == 1) -> DOWN(sem_r)	sem_r = 0	Lesen gesperrt
Interrupt		ankunft R1		
ZS 2	R1	1. DOWN(mutex_str)	mutex_str = 0	OK
		2. DOWN(sem_r)		Blockiert

W2 trifft während der zweiten Zeitscheibe ein und wird dadurch hinter W1 eingeordnet, sodass W1 vorher rechnen darf. -0,5

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
<i>Interrupt</i>		<i>ankunft W2</i>		
ZS 3	W2	1. DOWN(mutex_wc)		Blockiert

Analog wie bei a): Angabe der Schritte bis alle Prozesse fertig sind. -1

3/3 4.5

2/2 a)

```

) ls -la
dr-x----- joshua users  4 B Sat Dec 13 17:51:45 2025 .
dr-xr-xr-x joshua users  0 B Sat Dec 13 17:51:38 2025 ..
lr-x----- joshua users 64 B Sat Dec 13 17:51:47 2025 0 => /dev/null
lrwx----- joshua users 64 B Sat Dec 13 17:51:47 2025 1 => /dev/pts/0
l-wx----- joshua users 64 B Sat Dec 13 17:51:47 2025 2 => /tmp/err_uid
lrwx----- joshua users 64 B Sat Dec 13 17:51:47 2025 3 => /tmp/tempfileDTuKNe
) ls -la
dr-x----- joshua users  3 B Sat Dec 13 17:51:45 2025 .
dr-xr-xr-x joshua users  0 B Sat Dec 13 17:51:38 2025 ..
lr-x----- joshua users 64 B Sat Dec 13 17:51:47 2025 0 => /dev/null
lrwx----- joshua users 64 B Sat Dec 13 17:51:47 2025 1 => /dev/pts/0
l-wx----- joshua users 64 B Sat Dec 13 17:51:47 2025 2 => /tmp/err_uid

```



Unter `/proc/PID/fd/` befindet:

- 0: Verweist auf `/dev/null` – Standard Input.
- 1: Verweist auf Terminal (`/dev/pts/X`) – Standard Output.
- 2: Verweist auf `/tmp/err_uid` – Standard Error. ✓
- 3: Verweist auf die temp Datei `/tmp/tempfileXXXXXX` .
 - `mkstemp(str3)` erstellt einen Dateideskriptor. Da 0, 1 und 2 belegt sind, ist 3 der nächste freie.

1/1 b)

```

[nix-shell:/proc/52215/task]$ tree */fd
52215/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
├── 2 → /tmp/err_uid
└── 3 → /tmp/tempfileG48UVX
52216/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
├── 2 → /tmp/err_uid
└── 3 → /tmp/tempfileG48UVX

2 directories, 8 files

[nix-shell:/proc/52215/task]$ tree */fd
52215/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
└── 2 → /tmp/err_uid
52216/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
└── 2 → /tmp/err_uid

2 directories, 6 files

```

- Unterscheiden sich diese während der Ausführung des Programms?
 - Nein. Threads, die mit `pthread_create` erstellt werden, teilen sich (via `CLONE_FILES` Flag beim `clone`-Syscall)  dieselbe Dateideskriptortabelle.
 - Wenn man `/proc/PID/task/TID/fd` für den Hauptthread und den Worker-Thread ansieht, sind die Einträge identisch. 
- Fork statt Thread:
 - Wenn statt `pthread_create` ein `fork()` verwendet wird, würde der Kindprozess eine Kopie der Dateideskriptortabelle  erhalten.

- Wenn der Elternprozess dann `close(fp_ptr)` aufruft, wird nur sein Eintrag entfernt. ✓
- Der Kindprozess hätte den Dateideskriptor weiterhin offen und könnte erfolgreich schreiben. ✓
- => Prozesse haben getrennte Adressräume und Ressourcentabellen (Copy-on-Write bzw. Duplizierung beim Start), Threads teilen sich diese Ressourcen. ✓